

LoginService

Nota de Avaliacao do Desenho de Solucao

Arquitetura: LoginService -> Redis -> JWT

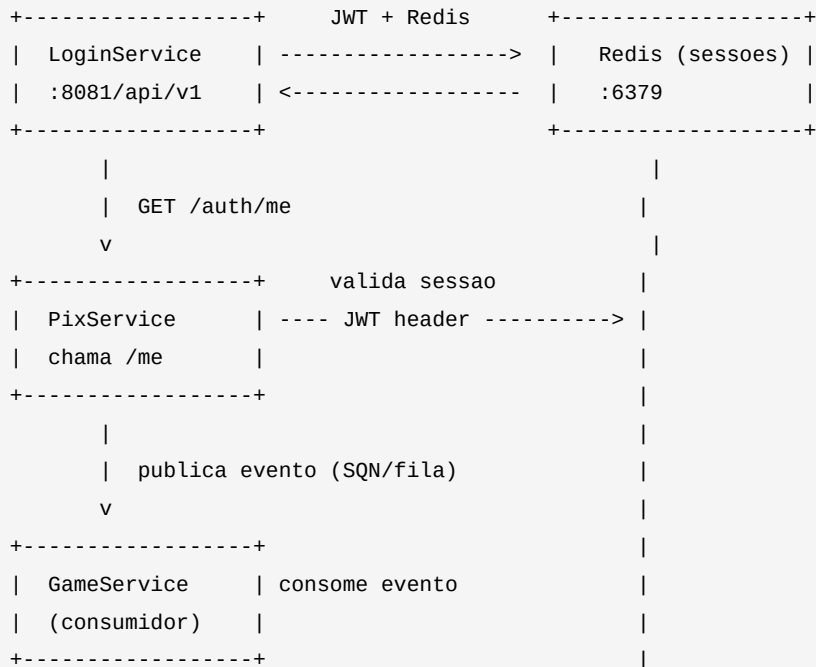
PixService -> /me -> SQN | GameService -> eventos

Itau - Data: 24 de April de 2026

1. Visao Geral da Arquitetura

O LoginService e um servico de autenticao baseado em Spring Boot (Java 21) que utiliza JWT (JSON Web Tokens) para autenticao stateless e Redis como store de sessoes. A arquitetura envolve tres servicos interdependentes: LoginService, PixService e GameService, comunicando-se por chamadas HTTP sincronas e eventos assincronos.

Diagrama de Arquitetura



2. Pontos Positivos

- Sessao stateless com JWT:** Arquitetura correta para microsservicos. O JWT carrega sessionId, role e contractService, evitando consultas desnecessarias ao banco em cada requisicao.
- Redis como store de sessao:** Uso inteligente do Redis para invalidacao de sessao sem depender de estado in-memory. Permite escalabilidade horizontal do LoginService.

- + **BCrypt para senhas:** Uso de BCryptPasswordEncoder e a pratica recomendada para hash de senhas, com resistencia a ataques de forza bruta.
- + **Spring Security + filtro JWT:** Integracao adequada com o ecossistema Spring Security, com filtros customizados para validacao JWT em cada requisicao.
- + **Threads virtuais (Java 21):** Uso de threads virtuais melhora a capacidade de concorrencia sem overhead de threads do sistema operacional.
- + **Separacao de responsabilidades:** Boa separacao entre controller, service, domain e repository, seguindo principios da arquitetura hexagonal.

3. Problemas Identificados e Recomendacoes

3.1. symmetricKey exposta no endpoint /me

CRITICO

Problema:

O endpoint GET /auth/me retorna o SessionDTO diretamente, incluindo o campo symmetricKey. Essa chave e um segredo interno gerado por sessao e nao deve ser exposta na camada de transporte.

Impacto:

Qualquer servico que chame /me (como o PixService) recebera essa chave simetrica. Se o trafego nao for criptografado (HTTPS ausente), essa chave pode ser interceptada. Viola o principio do menor privilegio.

Recomendacao:

Criar um MeResponseDTO sem o campo symmetricKey e retorna-lo no endpoint /me. Exemplo: public record MeResponseDTO(String sessionId, String username, Boolean contractService, String role) {}

3.2. JWT secret hardcoded no application.yaml

CRITICO

Problema:

O segredo JWT (ChangeThisSecretKeyForProdUseAtLeast32Chars!) esta hardcoded no application.yaml e no .env. Qualquer pessoa com acesso ao repositario pode forjar tokens JWT validos.

Impacto:

Comprometimento total da autenticao. Um atacante pode gerar JWTs validos para qualquer usuario.

Recomendacao:

Usar variavel de ambiente injetada pelo orquestrador (Kubernetes Secrets, AWS Secrets Manager, HashiCorp Vault). Nunca commitar segredos no repositario, mesmo em arquivos .env.

3.3. Ausencia de rate limiting no endpoint de login

ALTO

Problema:

O endpoint POST /auth/login nao possui protecao contra ataques de forza bruta. Nos testes realizados, foi possivel realizar 10+ tentativas consecutivas sem qualquer bloqueio.

Impacto:

Vulnerabilidade a ataques de forza bruta e credential stuffing, especialmente para o usuario Thiago com senha numerica simples (231299).

Recomendacao:

Implementar rate limiting com Bucket4j + Redis ou no nivel do API Gateway. Sugestao: maximo 5 tentativas por IP em 60 segundos.

3.4. Ausencia de HTTPS em producao

ALTO

Problema:

O servico nao possui configuracao de TLS/SSL. Todo o trafego, incluindo tokens JWT, e transmitido em texto claro.

Impacto:

Tokens JWT podem ser interceptados em ataques man-in-the-middle. O header Strict-Transport-Security (HSTS) tambem esta ausente.

Recomendacao:

Configurar TLS no Load Balancer/API Gateway que faz frente ao servico, ou configurar SSL diretamente no Spring Boot com `server.ssl.*`

3.5. PixService chama /me sem contrato de versionamento

ALTO**Problema:**

O PixService chama /me para obter informacoes do usuario autenticado. Se o contrato do endpoint /me mudar (renomear campos, remover symmetricKey), o PixService pode quebrar sem aviso.

Impacto:

Acoplamento forte entre PixService e LoginService. Uma mudanca no LoginService pode derrubar o PixService em producao.

Recomendacao:

Versionar o endpoint: /api/v1/auth/me -> /api/v2/auth/me. Criar contrato OpenAPI versionado. Implementar testes de contrato (Consumer-Driven Contract Testing com Pact).

3.6. Redis sem alta disponibilidade

ALTO**Problema:**

O Redis esta configurado como instancia unica (single-node). Toda a gestao de sessoes depende deste unico no.

Impacto:

Uma falha no Redis derruba toda a capacidade de autenticacao. Nenhum usuario conseguira fazer login ou ter sessoes validadas.

Recomendacao:

Configurar Redis em modo Sentinel (1 master + 2 replicas) ou Redis Cluster. Configurar `spring.data.redis.sentinel.*` ou `spring.data.redis.cluster.*`

3.7. GameService consome eventos sem rastreabilidade

MEDIO**Problema:**

O GameService consome eventos, mas o desenho de solucao nao especifica: mecanismo de transporte (Kafka, RabbitMQ, SQS?), schema dos eventos, versionamento, DLQ ou correlation ID.

Impacto:

Eventos perdidos, sem DLQ, podem causar perda silenciosa de dados. Sem correlation ID, e impossivel rastrear uma transacao end-to-end nos logs.

Recomendacao:

Definir mecanismo de mensageria (Kafka recomendado). Definir schema com Avro ou JSON Schema. Implementar DLQ. Incluir correlationId e traceId em todos os eventos.

3.8. Fluxo PixService -> /me -> SQN sem fallback

MEDIO**Problema:**

O PixService chama /me do LoginService de forma sincrona. Se o LoginService estiver indisponivel, toda a operacao PIX falha. Nao ha circuit breaker, retry ou fallback.

Impacto:

Falha em cascata: uma indisponibilidade do LoginService derruba o PixService.

Recomendacao:

Implementar circuit breaker com Resilience4j (@CircuitBreaker). Cache local no PixService para sessoes recentemente validadas (TTL curto). Retry com backoff exponencial.

3.9. SessionUtils com estado estatico mutavel

MEDIO

- Problema:**
A classe SessionUtils utiliza variaveis estaticas (static SessionService, JwtService) para manter referencias a beans Spring. Isso e anti-padrao e nao e thread-safe em cenarios de multiplos contextos Spring.
- Impacto:**
Condicoes de corrida em ambientes com multiplos context loaders. Dificulta testes unitarios isolados.
- Recomendacao:**
Eliminar o uso de estado estatico. Injetar SessionService e JwtService diretamente via construtor nas classes que precisam deles.

3.10. Dupla consulta ao banco no login

BAIXO

- Problema:**
Em LoginServiceImpl.login(), ha duas consultas ao banco: AuthenticationManager.authenticate() chama UserDetailsServiceImpl (findByUsername) e apos isso ha uma segunda chamada explicita.
- Impacto:**
Latencia desnecessaria no login e carga extra no banco de dados.
- Recomendacao:**
Usar o UserDetails retornado pela autenticao ou cachear o resultado da primeira consulta.

3.11. H2 Console e usuario inicial hardcoded

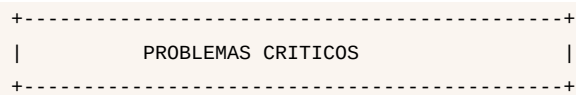
BAIXO

- Problema:**
O DataInitializer cria o usuario 'Thiago' com senha '231299' hardcoded. O H2 console esta habilitado no application.yaml.
- Impacto:**
Credenciais padrao conhecidas e console de banco expostos em producao.
- Recomendacao:**
Remover DataInitializer ou tornar condicional (@Profile('dev')). Desabilitar H2 console em producao (spring.h2.console.enabled: false).

4. Avaliacao por Dimensao

Dimensao	Nota	Justificativa
Autenticacao (JWT + Redis)	7/10	Correto e moderno, mas falhas criticas: secret hardcoded, symmetricKey exposta
Seguranca	5/10	Ausencia de rate limiting, HTTPS, HSTS e headers de seguranca
Resiliencia	5/10	Redis single-node, sem circuit breaker, sem DLQ
Rastreabilidade	4/10	Logs basicos presentes, mas sem correlationId nem tracing distribuido
Acoplamento entre servicos	5/10	PixService acoplado sincronamente ao LoginService sem fallback
Qualidade do codigo	7/10	Boa estrutura, mas SessionUtils estatico e dupla consulta ao banco
NOTA GERAL	5.5/10	Base solida com problemas criticos a corrigir antes de producao

5. Diagrama de Problemas Identificados



```

[LoginService] ---> [Redis (single-node)] ---> [JWT (secret hardcoded)]
    |                               |                               |
    |       SEM HA/Sentinel       |       NAO usar .yaml
    |
+----> POST /auth/login  <--- SEM rate limiting (forca bruta)
    |
+----> GET /auth/me      <--- EXPOE symmetricKey (CRITICO)
    |
    v
[PixService] ---> SQN <--- SEM circuit breaker / fallback
    |
    +-- Acoplamento sincrono rigido
        Se LoginService cair -> PIX cai

[GameService] <-- Consume eventos
    |
    +--> Mecanismo nao especificado
        SEM DLQ, SEM schema, SEM correlationId

```

6. Roadmap de Correcoes Recomendado

Sprint 1 - Bloqueadores para Producao

- [] Remover symmetricKey do response do /me -> criar MeResponseDTO
- [] Mover JWT secret para variavel de ambiente segura
- [] Implementar rate limiting no /auth/login (Bucket4j)
- [] Configurar HTTPS/TLS (no minimo no API Gateway)

Sprint 2 - Resiliencia e Seguranca

- [] Redis Sentinel ou Cluster
- [] Circuit breaker no PixService (Resilience4j)
- [] Headers de seguranca: HSTS, CSP, Referrer-Policy
- [] Remover DataInitializer ou tornar @Profile('dev')

Sprint 3 - Qualidade e Observabilidade

- [] Definir mecanismo de mensageria do GameService (Kafka + DLQ)
- [] Implementar tracing distribuido (OpenTelemetry + Jaeger/Zipkin)
- [] Contrato OpenAPI versionado + testes de contrato (Pact)
- [] Eliminar SessionUtils estatico
- [] Corrigir dupla consulta ao banco no login